ROAM: A Retrieval-augmented Optimization Agentic Model for Spontaneous Trip Blueprint Generation

Ye Zihao and Lin Xiaoheng

IDs: 59179849, 59374250

Emails: ye.zihao@my.cityu.edu.hk, xiaohelin2-c@my.cityu.edu.hk

Abstract

The post-pandemic surge in spontaneous short trips has intensified the need for automated travel planning that transforms vague natural language preferences into physically executable multi-day itineraries. Existing large language model (LLM)-based tools often produce plans that violate spatial distance constraints and attraction operating hours. We present ROAM, a Retrieval-augmented Optimization Agentic Model that fully automates itinerary generation from a single user sentence. ROAM follows a three-stage pipeline: (1) semantic retrieval via vector similarity over a database of 515 real-world POIs across four Chinese cities; (2) spatial optimization employing OSRM-based real driving distances, K-means cross-day clustering, simulated annealing TSP solving, and a greedy time-window correction algorithm that ensures minute-level feasibility; (3) generation enhancement that injects optimized routes, real-time weather, and category knowledge into an LLM to produce structured JSON and interactive HTML maps. Compared to LLM-only baselines and prior single-day systems like ITINERA, ROAM uniquely addresses multi-day load balancing, hard time-window constraints, and real-world routing. Evaluation demonstrates significant improvements in spatial reasonableness, time compliance rate, and semantic matching, compressing hours of manual planning into minutes and enabling truly spontaneous travel.

Team Member Contribution

Ye Zihao (Team Leader) assumed overall responsibility for the entire project, including problem formulation, algorithm design, code development, experimentation, and paper preparation.

Lin Xiaoheng (Team member) contributed to preliminary user testing, basic formatting, and reference collection.

Introduction

The history of tourism is almost as old as the history of human travel. In 1841, Thomas Cook organized the first group train trip, marking the birth of the modern travel agency industry and the commercialization of travel. In the mid-20th century, with the global expansion of civil aviation and highway networks, the era of mass tourism officially began, and paper travel guides such as the Michelin Guide and Lonely Planet became the primary sources of information for independent travelers. The rise of the internet fundamentally reshaped the tourism industry. In 1999, Ctrip was founded, marking the beginning of online tourism in China, followed by the emergence of platforms such as Qunar and eLong, which moved the booking of flights, hotels, and attractions from offline to online. In the 2010s, the widespread adoption of mobile internet and smartphones gave rise to user-generated content (UGC) communities such as Mafengwo and Qiongyou. The vast volume of travelogues, attraction reviews, and route sharing essentially resolved the problem of information scarcity. However, information overload and decision-making complexity have since escalated sharply. An ordinary traveler planning a weekend short trip often needs to navigate multiple platforms, read dozens of travelogues, and manually cross-check attraction opening hours, commuting distances, and weather conditions, with the entire preparation process taking three to five hours. The prominent predicament of travel planning lies in information overload, which makes it difficult to efficiently integrate fragmented information into a coherent and actionable itinerary.

In early 2020, the COVID-19 pandemic pressed the pause button on cross-border travel. Border controls and social distancing measures implemented by various countries brought international travel to a near standstill, making domestic short-distance trips an alternative. During the pandemic, frequent attraction closures, adjustments to transportation schedules, and dynamic changes in health-code and nucleic-acid testing policies amplified both the rigidity and uncertainty of travel planning—static travel guides became nearly ineffective in the face of rapidly shifting regulations.

In the post-pandemic era, China's cultural tourism industry has experienced a strong recovery. Weekend short-distance trips and holiday travel have become the mainstream leisure modes for urban residents. However, the planning approaches that support this enormous travel demand have not been upgraded accordingly. The social landscape of the post-pandemic era presents notable dual characteristics: on the one hand, workplace competition has intensified, "involution" has spread, and overtime work has become the norm; on the other hand, people's yearning for a better quality of life and for "poetry and faraway places" has grown even stronger. This tension gives rise to a common predicament: most working people desire weekend getaways to relax, yet due to heavy work pressure and limited energy, they are unable to plan their itineraries in advance. They want a trip that can be taken on a whim, but in reality, without spending hours on information gathering and scheduling, such a spontaneous trip often means "arriving only to be at a loss."

Current mainstream travel tools fall short in addressing this demand. Platforms such as Ctrip and Mafengwo are essentially "search-plus-list" information aggregation models—they can tell users which attractions are popular in a city, but they cannot perform spatiotemporal scheduling that spans multiple attractions. Recently emerged large language model chatbots like ChatGPT and Kimi, although capable of generating draft itineraries based on natural language queries, have been shown in practice to suffer from three systematic deficiencies. First, spatial cognitive bias: they significantly underestimate actual commuting time between city attractions, potentially describing two attractions that are eight kilometers apart as "within walking distance." Second, disregard for time constraints: they lack a hard awareness of attractions' operating time windows and frequently schedule visits after closing hours. Third, knowledge lag: they are unable to perceive real-time weather, temporary closures, or road construction. These deficiencies render itineraries generated solely by large language models highly unreliable when executed in the physical world—a seemingly reasonable plan can completely collapse at the very first stop because of "Monday closure" or "arriving after closing time."

This is precisely the core value of Agentic AI. A pure LLM is a "thinker"—it can understand semantic needs such as "I want to experience the history and culture," but it cannot by itself check weather, calculate distances, or verify opening hours. An Agentic system, on the other hand, is an "actor"—it can proactively invoke external tools, including weather APIs, route planning engines, geographic databases, and more, to jointly reason over semantic understanding and physical constraints, and output a plan that is temporally and spatially executable. Travel planning inherently requires a closed loop of "perception, reasoning, and action": perceiving real-world weather and traffic conditions, reasoning about the temporal sequencing and load among attractions, and acting to generate an itinerary that users can directly use. Therefore, the Agentic paradigm is not merely an enhancement but a critical leap from "unusable" to "usable."

Based on the above observations, this paper proposes and implements ROAM (Retrieval-Augmented Optimization Agent for Multi-Day Travel Planning), a retrieval-augmented optimization agent for multi-day short-distance travel. ROAM has a clear and focused design goal: allowing users to input their travel requirements in a single natural-language sentence, and enabling the system to fully automate the entire process from point-of-interest retrieval and spatial route optimization to itinerary narrative generation, compressing the manual planning that originally took hours into minutes. The system adopts a three-stage pipeline architecture. The first stage is semantic retrieval, which uses an embedding model to match user preferences against a point-of-interest database containing 515 real-world attractions via vector similarity, retrieving a candidate set of attractions. The second stage is spatial optimization, which employs the open-source routing engine OSRM to obtain real-world road distance matrices, and generates daily optimal visit sequences and minute-level timelines through K-means cross-day clustering, simulated annealing for the traveling salesman problem, and a hard time-window correction algorithm based on greedy exchanges. The third stage is generation enhancement, which injects dynamic context—including the optimized spatiotemporal route, real-time weather forecasts from wttr.in, and common-sense knowledge about attraction categories—into the prompts of a large language model, producing a multi-day itinerary narrative that combines both route optimality and linguistic fluency. The final output includes structured JSON data and an interactive HTML map based on Amap (Gaode) tiles.

The scope of this project is defined as follows: it focuses on automated planning for short-duration multi-day itineraries of up to seven days, covering 515 real-world points of interest (POIs) across four cities: Fuzhou, Guangzhou, Shanghai, and Shenzhen. It supports users in inputting their travel preferences in natural language, and the system fully automates the end-to-end process from retrieval and optimization to generation. The system is not intended to replace human travel planners, but rather serves as an "intelligent travel assistant," providing efficient and reliable automated support during the planning phase when users are most pressed for time and energy, turning "a trip on a whim" from a mere slogan into an actionable service.

Prior Works and Limitations

Existing commercial platforms and conversational assistants.
Current mainstream travel planning tools can be categorized into two types. One type is search-aggregation platforms, represented by Ctrip and Mafengwo, which list attractions, hotels, and transportation information in list or map form. They can answer "what are the popular attractions in a city," but cannot automatically perform spatiotemporal scheduling across multiple points and multiple days. The other type is LLM-based conversational assistants, represented by ChatGPT and Kimi. Although they can generate draft itineraries based on natural language, they often suffer from problems such as miscalculated commuting times and violations of operating time windows—issues that appear plausible but are actually infeasible—due to a lack of spatial distance computation and real-time data access (Xie et al., 2024). Neither of these two types of tools has bridged the gap between "vague semantic requirements" and "physically executable plans."

Traditional Operations Research and Recommendation Systems.
In contrast, academic research often models travel route planning as the Orienteering Problem (OP) or the Traveling Salesman Problem with Time Windows (TSPTW), and solves them using heuristic methods such as genetic algorithms and ant colony optimization. These approaches rely heavily on manually defined attraction scores and cannot capture complex preferences expressed in natural language, such as "having an artistic vibe" or "being family-friendly." In the meantime, classical recommendation systems (e.g., collaborative filtering, matrix factorization) only output isolated lists of points of interest, lacking the ability to generate spatiotemporally coherent day-by-day itineraries.

ITINERA and Open-Domain Urban Itinerary Planning.
Most directly related to this project is the ITINERA system proposed by Tang et al. (2024), which for the first time formalized the Open-domain Urban Itinerary Planning (OUIP) problem and designed a complete pipeline consisting of "user POI database construction → request decomposition → preference retrieval → clustering-aware spatial optimization (CSO) → itinerary generation." Experiments demonstrated that this architecture significantly outperforms pure LLM baselines in terms of recall and spatial coherence. However, ITINERA has three key limitations, which precisely motivate this research:
1.Scenario constraints: It is primarily designed for single-day urban strolls; its clustering and hierarchical TSP only optimize the order for a single day, without addressing cross-day load balancing or dynamic accommodation matching for multi-day itineraries.
2.Soft time constraints: The time budget is entirely estimated by the LLM, lacking hard verification of attractions' absolute operating time windows (e.g., closing at 17:00), leading to frequent violations such as "arriving after closing" during actual execution.
3.Fixed dwell times: It does not model visit durations differently by attraction category (e.g., 2 hours for museums vs. 30 minutes for viewing platforms), causing inaccurate arrival time calculations.

Positioning and differentiation of this project.
The above limitations have become increasingly prominent in the post-pandemic era—users demand not only higher personalization and real-time accuracy, but also balanced daily itinerary loads and absolute physical feasibility. To overcome these shortcomings, ROAM extends the OUIP task to multi-day scenarios, mandatorily incorporates a real-world road distance matrix (OSRM), and designs post-optimization correction heuristics with time-window constraints. Unlike ITINERA's "LLM estimation + sequential generation" approach, our method relies on "precise computation + hard correction," systematically ensuring the transition from "semantically reasonable" to "temporally compliant," while also filling the gap left by commercial products in end-to-end automated planning.

Proposed Method/Product

6.1 System Overview

ROAM is a retrieval-augmented, spatiotemporally aware agent system for multi-day travel planning. As illustrated in Figure 1, the system adopts a three-stage pipeline architecture: "semantic retrieval → spatial optimization → generation enhancement." After the user inputs travel requirements in natural language, the system first retrieves semantically matching candidate attractions from the POI database via vector retrieval. It then incorporates a real-world road distance matrix and performs cross-day route optimization using clustering algorithms and a Traveling Salesman Problem (TSP) solver with time-window constraints. Finally, the optimized spatiotemporal route, along with real-time weather and operating hours, is injected into the prompts of a large language model (LLM) to generate an itinerary that is both physically feasible and linguistically expressive.

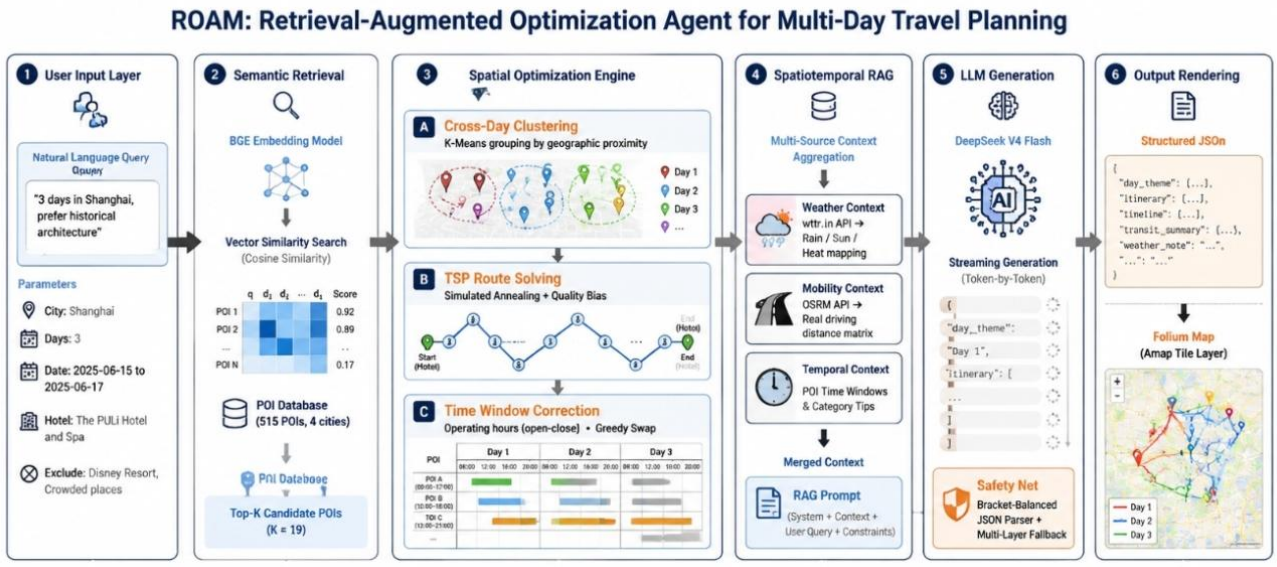


Figure 1: ROAM system architecture and data flow diagram.

6.2 System Core Components and Agentic Workflow

6.2.1 User Input Layer

Users describe their requirements in natural language. The system first invokes an LLM to parse the natural language into structured requirements, extracting positive preferences, negative constraints, and must-visit attractions, which serve as inputs for subsequent retrieval.

6.2.2 Semantic Retrieval and Filtering

The retrieval module employs a Hugging Face model to encode the text descriptions from the POI database into normalized vectors. Meanwhile, the system calls a weather API to obtain daily weather forecasts for the travel period, providing real-time environmental awareness for the subsequent spatial optimization and generation stages.

6.2.3 Spatial Optimization Engine

Spatial optimization is the core module that distinguishes ROAM from pure LLM-based approaches, and it consists of three sub-stages:

1. Cross-day clustering. First, a graph-based maximum clique algorithm is used to perform fine-grained spatial clustering on the candidate POIs. Then, K-Means is applied to cluster the centroids of these fine-

grained clusters, merging them into “day groups” equal to the number of itinerary days. A greedy transfer algorithm is used to balance the number of POIs across groups, ensuring load balance for each day.
2. TSP route solving. For the POI set of each day, real-world driving distances between any two points are calculated by calling an API. Subsequently, a simulated annealing algorithm is run to solve the Traveling Salesman Problem, yielding a visit order that is spatially shortest (or near-shortest).
3. Hard time-window constraint correction. The initial route obtained from the TSP solver does not guarantee compliance with each attraction’s operating time window. Starting from the departure time, the system sequentially accumulates commuting time and dwell time, and checks whether each arrival time falls within the [open, close] interval. This module ensures that the output route is executable at the minute level in the physical world.

6.2.4 Spatiotemporal RAG Context Generation

Traditional LLMs face three major deficiencies in itinerary generation: knowledge cutoff, weak spatial awareness, and factual hallucination. ROAM’s spatiotemporal RAG module packages three types of dynamic context—weather context, POI dynamic context, and transportation context—into structured text, which is then injected into the LLM prompts.

6.2.5 Itinerary Generation and Visualization Output

The generation module assembles the above content into prompts and submits them to the LLM to generate the daily itinerary. The generation adopts a streaming output mode, which can display the LLM's reasoning process token by token in real time, facilitating debugging and user experience. The system uses the Folium library to generate interactive HTML maps, producing an independent route map for each planned day, as well as an overview comparison map.

Table 1: External tools and data sources used by the ROAM system.

External tools	Purpose	Invocation method	Fallback Strategy
wttr.in	Weather forecast	HTTP Response	Omit weather context
OSRM	Driving distance and duration	HTTP Response	Compute straight-line (Haversine) distance
Amap API	POI geocoding	REST API	Use pre-stored coordinate cache
	Attraction rating retrieval		static rating data
	Base map on HTML	Tile URL	OpenStreetMap
BGE-small-zh	POI text embedding	sentence-transformers	Switch to a lighter local embedding model
DeepSeek	Requirement parsing + itinerary generation	OpenAI API Calling	LLMs from Ollama

6.3 Evaluation Scheme

To systematically verify the contribution of each component of ROAM to the quality of multi-day itinerary planning, this study evaluates the model performance from three dimensions: (1) spatial reasonableness: the extra detour ratio of the LLM-generated route compared to the TSP optimal solution; (2) time compliance rate: the proportion of attractions in the itinerary that satisfy the operating time

windows; (3) semantic matching degree: the average cosine similarity between the output attractions and the user preference embedding vectors.

6.3.2 Baseline Comparison: ROAM vs. Bare LLM

This experiment aims to answer the core question: whether the ROAM agent system outperforms the LLM that directly generates itineraries. Therefore, we compare the bare LLM baseline (LLM – Only) with the complete ROAM system. The bare LLM baseline consists only of the complete user requirement prompt, the city name, and the number of days, without any contextual information.

6.3.3 Ablation Experiments

To quantify the independent contribution of each component to system performance, this study removes five modules from the complete ROAM system—the retrieval module, the spatial optimization module, the time-window correction module, the RAG context module, and the quality bias module—resulting in five ablation variants, each removing only a single component while keeping the rest unchanged.

Expected Deliverables

This project encompasses both engineering and research aspects. The core deliverables include an interactive travel planning agent (ROAM), specifically comprising a lightweight web-based interactive interface built with Streamlit, a structured itinerary JSON file strictly adhering to a JSON schema, and accompanying interactive map visualizations. Through the front-end page built with Streamlit, users simply input the target city and natural language preferences (e.g., "Shenzhen, culture and cuisine"). The model performs semantic analysis and decomposition of the prompt, completes retrieval, reasoning, and planning, and generates five files:

1. A downloadable JSON itinerary file; this JSON file must comprehensively include the itinerary title, daily theme summaries, coordinates and descriptions of each POI, transportation modes and estimated costs between points, weather integration hints, and a detailed timeline.
2. Automated rendering of multiple maps: the system renders an overview HTML map with daily itinerary points marked in different colors, as well as individual day-by-day HTML maps that independently render and display the route and POI details for each day.
3. Construction of datasets for four different cities; the team will also conduct model comparison experiments and module ablation experiments on the datasets, to verify whether the designed agent and modules possess verifiability and generalization capability.
4. Code will be available at https://github.com/yzhzy12321/travel-agent/
5. Presentation slides, highlighting how the team designed the agent architecture, and the principles and implementation of reasoning under vague requirements.

Disclosure of the use of AI

The original draft was written entirely by the team members in Simplified Chinese. Figure 1, the system architecture diagram, was generated by ChatGPT's image generation capability (ChatGPT2Image) based on a sketch and description provided by the team. Deepseek V4 Pro was used solely for English translation and grammatical polishing of the final text. All technical ideas, system design, evaluation plans, and prior work analysis were formulated by the human authors, based on the academic references cited in this proposal and the prior work we analyzed above.

References

[1] Lewis P, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.
[2] Yao S, et al. ReAct: Synergizing reasoning and acting in language models[C]//Proceedings of the 11th International Conference on Learning Representations (ICLR), 2023.
[3] Laporte G. The traveling salesman problem: An overview of exact and approximate algorithms[J]. European Journal of Operational Research, 1992, 59(2): 231-247.
[4] Xie J, et al. TravelPlanner: A benchmark for real-world planning with language agents[EB/OL]. (2024). arXiv:2402.01622. [2026-06-23].
[5] Tang Y, et al. ITINERA: Integrating spatial optimization with large language models for open-domain urban itinerary planning[EB/OL]. (2024). arXiv:2402.07204. [2026-06-23].
[6] Sharma A, Gani A, Asirvatham D, et al. Travel behavior modeling: Taxonomy, challenges, and opportunities[J]. International Journal of Advanced Computer Science and Applications, 2021, 12(5): 774-786.
[7] Backhaus C, Heussler T, Croce V. Planning horizon in international travel decision-making: The role of individual and cultural determinants[J]. Journal of Travel Research, 2023, 62(2): 432-447.
[8] Marzal E, Sebastia L. Solving the tourist trip design problem with time windows and variable profit[J]. Applied Soft Computing, 2024, 155: 111399.
[9] Winsa A, Barthelmes L, Alpers S, et al. Personalized day-trip planning: A TSP-TW-based multimodal multicriteria optimisation approach[C]//Proceedings of the 15th International Conference on Ambient Systems, Networks and Technologies (ANT). Hasselt, Belgium: Elsevier, 2024: 352-360.
[10] Jayasuriya N, Sumanathilaka D. A systematic decade review of trip route planning with travel time estimation based on user preferences and behavior[EB/OL]. (2025-03-30). arXiv:2503.23486. [2026-06-23].
[11] Sigala M. Tourism and COVID-19: Impacts and implications for advancing and resetting industry and research[J]. Journal of Business Research, 2020, 117: 312-321.
[12] Amap Open Platform. Route planning and geocoding API documentation[EB/OL]. [2026-06-23]. https://lbs.amap.com/.
[13] OpenStreetMap Contributors. OSRM project documentation[EB/OL]. [2026-06-23]. https://project-osrm.org/.
[14] Chubin I. wttr.in: Command-line weather service[EB/OL]. [2026-06-23]. https://wttr.in/.
[15] Chen A, Ge X, Fu Z, Xiao Y, Chen J. TravelAgent: An AI assistant for personalized travel planning[EB/OL]. (2024). arXiv:2409.08069. [2026-07-05].
[16] Yang D, Lu C, Wang Q, Ma X, Gao Y, Hu Y, Zhao H. Plan your travel and travel with your plan: Wide-horizon planning and evaluation via LLM[EB/OL]. (2025). arXiv:2506.12421. [2026-07-05].
[17] Shao J J, Zhang B W, Yang X W, et al. ChinaTravel: An open-ended benchmark for language agents in Chinese travel planning[EB/OL]. (2025). arXiv:2412.13682. [2026-07-05].
[18] Chaudhuri S, Purkar P, Raghav R, Mallick S, Gupta M, Jana A, Ghosh S. TripCraft: A benchmark for spatio-temporally fine grained travel planning[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Vienna, Austria: ACL, 2025: 17035-17064.
[19] Wang K, Shen Y, Lv C, Zheng X, Huang X. TripTailor: A real-world benchmark for personalized travel planning[C]//Findings of the Association for Computational Linguistics: ACL 2025. Vienna, Austria: ACL, 2025: 9705-9723.
[20] Qu Y, et al. TripScore: Benchmarking and rewarding real-world travel planning with fine-grained evaluation[EB/OL]. (2025). arXiv:2510.09011. [2026-07-05].

[21] Oh J, et al. Flex-TravelPlanner: A benchmark for flexible planning with language agents[EB/OL]. (2025). arXiv:2506.04649. [2026-07-05].
[22] Chen J, et al. Looking ahead to avoid being late: Solving hard-constrained traveling salesman problem[EB/OL]. (2024). arXiv:2403.05318. [2026-07-05].
[23] Singh A, et al. Agentic retrieval-augmented generation: A survey on Agentic RAG[EB/OL]. (2025). arXiv:2501.09136. [2026-07-05].
[24] Feng J, et al. A survey of large language model-powered spatial intelligence across scales: Advances in embodied agents, smart cities, and earth science[EB/OL]. (2025). arXiv:2504.09848. [2026-07-05].